Zenopher: Backend (Free-Tier MVP)

Status: Draft v1 **Replaces:** `zenopher-backend-mvp.md` for the free setup. Where this document differs, it wins. **Relates to:** `zenopher-backend.md` (full design) and `zenopher-start-page.md` (front-end flows).

---

1. Summary

The backend runs entirely on free tiers. It has four jobs:

1. Accounts and sessions
2. Storing encrypted designs and assets, with version checks
3. Sharing, trash, export, and deletion
4. The AI critique, through the Muse API, funded by 150 credits

Everything heavy (rendering, thumbnails, PSD parsing, rules checks) runs in the user's browser. The server stores and checks; it does not process pixels. This is the main design decision that makes the free tier workable.

---

2. Platform

Need	Choice	Why
API	**Cloudflare Workers**, TypeScript	No idle sleep, free plan, native fit with R2 and D1
Database	**Cloudflare D1** (SQLite)	Free tier, same account as Workers, no connection pooling to manage
Object storage	**Cloudflare R2**	Free tier, no egress fees
Scheduled jobs	**Cron Triggers**	Free, runs Workers on a schedule
Secrets	**Workers secrets**	Encrypted by the platform, never in code
Bot protection	**Cloudflare Turnstile**	Free, on sign-up and sign-in
Rate limiting	**Workers Rate Limiting** if available on the plan, otherwise D1 counters	Verify on the current plan
Frontend	**Cloudflare Pages** (preferred) or Vercel Hobby	Vercel Hobby is for non-commercial use only
Email	Free tier of a transactional email service, over HTTPS	Daily send limit
Model	**Muse API**, from the 150-credit balance	See Section 9

Check each product's current free-tier limits before relying on them. The constraints in Section 3 are the ones that shape the design.

---

3. Free-tier constraints that shape the design

These limits change, so verify the current values. The design is built around the kinds of limit listed here, not the exact numbers.

Constraint	Effect on the design
Low CPU time per request on the free Workers plan	No image decoding, re-encoding, PSD parsing, or rendering on the server. The browser does this work.
Limited subrequests per request	Keep each request's outbound calls small. No fan-out.
Daily request cap	Autosave and polling must be efficient. Client-side debouncing is required.
D1 daily read and write caps, and a storage cap	Batch writes. Keep rows small. Store document content in R2, not D1.
No long-running work in requests	Anything slow (exports, purges, retries) runs from cron, in small batches.
Request body size limits	Uploads are capped well below the platform limit. Large uploads are refused early.

If a design needs more CPU than the free plan allows, the answer is to move that step into the browser, not to upgrade by default.

4. Architecture

One codebase, two entry points: `fetch` for requests and `scheduled` for cron.

---

5. Division of work: browser vs. server

Task	Where	Notes
Canvas rendering, filters, blend modes	Browser (WASM)	
Rules checks and style pattern measurements	Browser (WASM)	Work offline
Confidence levels, suggestions, fixes, Apply all	Browser	
Thumbnail generation	Browser	Client uploads the thumbnail PNG. Server checks the signature and size only.
PSD import	Browser	The engine parses the PSD locally and saves the result as `.zph`. The PSD itself is not uploaded.
PNG and JPEG import	Browser	Client re-encodes before saving. Server still validates the type and size.
Export to PNG, JPEG, PSD	Browser	
Critique image preparation	Browser	Client renders the page as PNG at a fixed maximum size
Encryption of design content	Server	Required for the envelope design (Section 7)
Validation, auth, budget, storage	Server	

Consequence for security: the server cannot re-encode uploads on the free plan, so it relies on strict type checks, size limits, and safe serving headers (Section 8.4). Client-side processing is hygiene, not a security boundary. The server treats every upload as hostile, whatever the client did.

---

6. Data model (D1)

```sql
```

```sql
CREATE TABLE users (
  id             TEXT PRIMARY KEY,           -- random 128-bit, base32
  email          TEXT NOT NULL UNIQUE,       -- lowercased
  email_verified INTEGER NOT NULL DEFAULT 0,
  display_name   TEXT NOT NULL,
  status         TEXT NOT NULL DEFAULT 'active',   -- active, pending_deletion,
  deletion_at    INTEGER,                    -- unix seconds, end of grace perio
  created_at     INTEGER NOT NULL,
  updated_at     INTEGER NOT NULL
);

CREATE TABLE oauth_links (
  provider       TEXT NOT NULL,              -- 'google'
  subject        TEXT NOT NULL,
  user_id        TEXT NOT NULL REFERENCES users(id),
  PRIMARY KEY (provider, subject)
);

CREATE TABLE sessions (
  id             TEXT PRIMARY KEY,
  user_id        TEXT NOT NULL REFERENCES users(id),
  family_id      TEXT NOT NULL,              -- for refresh-reuse detection
  refresh_hash   TEXT NOT NULL,              -- SHA-256 of the current refresh t
  device         TEXT,
  created_at     INTEGER NOT NULL,
  last_used_at   INTEGER NOT NULL,
  expires_at     INTEGER NOT NULL,
  revoked_at     INTEGER
);

CREATE TABLE auth_tokens (                   -- magic links and verification
  token_hash     TEXT PRIMARY KEY,           -- SHA-256
  user_id        TEXT,
  purpose        TEXT NOT NULL,              -- 'signin', 'verify'
  email          TEXT NOT NULL,
  expires_at     INTEGER NOT NULL,
  used_at        INTEGER
);

CREATE TABLE designs (
  id             TEXT PRIMARY KEY,           -- client-generated UUID, validated
  owner_id       TEXT NOT NULL REFERENCES users(id),
  title          TEXT NOT NULL,
  mode           TEXT NOT NULL,              -- creative, corporate
```

```
  style_id      TEXT,                       -- catalog ID and version, e.g. 'ba
  width_px      INTEGER NOT NULL,
  height_px     INTEGER NOT NULL,
  current_version INTEGER NOT NULL,
  wrapped_dek   BLOB NOT NULL,              -- per-design key, wrapped by the m
  trashed_at    INTEGER,
  created_at    INTEGER NOT NULL,
  updated_at    INTEGER NOT NULL
);
CREATE INDEX designs_owner_updated ON designs(owner_id, updated_at);
CREATE INDEX designs_trash ON designs(trashed_at);

CREATE TABLE design_versions (
  design_id     TEXT NOT NULL REFERENCES designs(id),
  version       INTEGER NOT NULL,
  r2_key        TEXT NOT NULL,              -- encrypted blob
  byte_size     INTEGER NOT NULL,
  content_hash  TEXT NOT NULL,              -- SHA-256 of the plaintext, checke
  created_at    INTEGER NOT NULL,
  PRIMARY KEY (design_id, version)
);

CREATE TABLE assets (
  id            TEXT PRIMARY KEY,
  owner_id      TEXT NOT NULL REFERENCES users(id),
  r2_key        TEXT NOT NULL,
  mime          TEXT NOT NULL,              -- image/png, image/jpeg, font type
  byte_size     INTEGER NOT NULL,
  status        TEXT NOT NULL,              -- pending, ready, rejected
  created_at    INTEGER NOT NULL
);

CREATE TABLE share_grants (
  id            TEXT PRIMARY KEY,
  design_id     TEXT NOT NULL REFERENCES designs(id),
  grantee_user_id TEXT,                     -- null for link shares
  token_hash    TEXT,                       -- link shares only
  permission    TEXT NOT NULL,              -- view, comment
  expires_at    INTEGER,
  revoked_at    INTEGER,
  created_at    INTEGER NOT NULL
);

CREATE TABLE consents (                     -- append-only
  id            INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
  user_id       TEXT NOT NULL REFERENCES users(id),
  purpose       TEXT NOT NULL,              -- 'critique', 'learning'
  decision      TEXT NOT NULL,              -- 'granted', 'withdrawn'
  policy_version TEXT NOT NULL,
  created_at    INTEGER NOT NULL
);

CREATE TABLE learning_events (
  pseudo_id     TEXT NOT NULL,              -- random per user, not the user ID
  command_id    TEXT NOT NULL,
  mode          TEXT NOT NULL,
  style_id      TEXT,
  outcome       TEXT NOT NULL,              -- applied, dismissed, undone
  occurred_at   INTEGER NOT NULL
);
CREATE INDEX events_time ON learning_events(occurred_at);

CREATE TABLE learning_link (                -- separate from events
  user_id       TEXT PRIMARY KEY REFERENCES users(id),
  pseudo_id     TEXT NOT NULL UNIQUE
);

CREATE TABLE budget (                       -- single row
  id            INTEGER PRIMARY KEY CHECK (id = 1),
  starting_credits REAL NOT NULL,           -- set in config, corrected weekly
  spent_credits REAL NOT NULL DEFAULT 0,
  reserved_credits REAL NOT NULL DEFAULT 0,
  reserve_credits REAL NOT NULL,            -- never spent on user reviews
  updated_at    INTEGER NOT NULL
);

CREATE TABLE model_usage (
  request_id    TEXT PRIMARY KEY,
  user_id_hash  TEXT NOT NULL,
  reserved      REAL NOT NULL,
  actual        REAL,
  status        TEXT NOT NULL,              -- reserved, complete, released, fa
  created_at    INTEGER NOT NULL
);

CREATE TABLE critique_cache (
  image_hash    TEXT PRIMARY KEY,
  result_text   TEXT NOT NULL,
  created_at    INTEGER NOT NULL
);
```

```
CREATE TABLE jobs (
  id            INTEGER PRIMARY KEY AUTOINCREMENT,
  kind          TEXT NOT NULL,              -- export, delete_account, delete_l
  user_id       TEXT NOT NULL,
  status        TEXT NOT NULL,              -- queued, running, done, failed
  attempts      INTEGER NOT NULL DEFAULT 0,
  run_after     INTEGER NOT NULL,
  created_at    INTEGER NOT NULL
);

CREATE TABLE rate_counters (
  key           TEXT PRIMARY KEY,           -- route:scope:hashed-id
  window_start  INTEGER NOT NULL,
  count         INTEGER NOT NULL
);

CREATE TABLE idempotency (
  key           TEXT PRIMARY KEY,           -- user + client-supplied key
  response      TEXT NOT NULL,
  created_at    INTEGER NOT NULL
);
```

Notes:

- **IDs** are random, not sequential, so they can't be guessed or counted.
- **Timestamps** are integer unix seconds, so comparisons and indexes stay simple.
- **Document content is never in D1.** Only the R2 key and hash are stored.
- **Table names and columns** are fixed in the migration files. Queries use bound parameters only.

---

7. Encryption

7.1 Scheme

```
Master key (32 bytes, Workers secret: ZPH_MASTER_KEY)
   | wraps (AES-256-GCM, random 96-bit nonce, AAD = design ID)
   ▼
Per-design data key (DEK, 32 bytes random)   stored in designs.wrapped_dek
   | encrypts (AES-256-GCM, random 96-bit nonce per blob, AAD = design ID +
version)
   ▼
Version blob in R2                            nonce prepended to ciphertext
```

- Web Crypto provides AES-GCM. No custom crypto.
- Nonces are random per encryption. With a 96-bit nonce, the safe limit per key is far above what a design will ever produce.
- AAD binds each blob to its design and version. A blob moved to another design or version fails to decrypt.
- Blobs are compressed with gzip (`CompressionStream`) before encryption.

7.2 Crypto-shredding

Deleting a design's `wrapped_dek` row makes every blob for that design unreadable, including copies in weekly exports. Account deletion removes the wrapped keys for all of that user's designs in one batch.

7.3 What this protects and what it doesn't

Protects against: leaked R2 buckets, leaked D1 data, leaked exports, and anyone with database access but not the Worker secret.

Does not protect against: a compromised Worker, which can read the master key. Anyone who controls the Worker can decrypt for the owner, and this is true of any server-held key.

Say "encrypted on our servers, with keys held separately from the data." Don't say "only you can read your designs."

7.4 Master key rotation

Store the key version alongside each wrapped DEK. Adding a new master key version and re-wrapping DEKs is a cron job over `designs`. Design blobs don't change, because only the DEK wrapping changes.

---

8. Security

8.1 Authentication

- **Primary sign-in: magic link by email, and Google.** No passwords are stored in the free MVP. This avoids password hashing, which is CPU-heavy and hard to fit into the free plan's CPU limit, and removes the largest class of account-takeover risk.
- **Magic links:** random 256-bit tokens, stored hashed, valid for 15 minutes, single use. The response is identical whether or not the email exists.
- **Google sign-in:** authorization code flow with PKCE and a `state` value. The Worker verifies the ID token's signature, issuer, audience, and expiry.
- **Passwords can be added later** with a hashing approach that fits the platform's CPU limits, after checking that it does. Don't add them without that check.

8.2 Sessions

- **Access token:** a short-lived signed token (15 minutes), with user ID and session ID only. Verified on each request. Revocation is checked against D1 for sensitive

operations.

- **Refresh token:** opaque, 256-bit, stored as a hash in `sessions`. Rotated on each use. A reused refresh token revokes the whole `family_id`.
- **Cookies:** refresh token in an HttpOnly, Secure, SameSite=Strict cookie scoped to the auth path.
- **Sign-out** revokes the session row. Revoking all sessions is a single update.

8.3 Authorization

Every query on designs, versions, assets, shares, and critique filters by owner or an active grant in the query itself. Not-found and forbidden both return `404`.

Authorization tests run against every endpoint on every change, as a release gate. The same matrix as the full backend design applies (`zenopher-backend.md`, Section 5.3).

8.4 Uploads and serving

Without server-side re-encoding, the risk is a file that is one type but served as another. Controls:

- **Accept only:** PNG, JPEG, WebP, `.zph` (gzip-compressed JSON), and the font types listed in the catalog. Reject everything else, including SVG and HTML.
- **Check the magic bytes** of the upload against the declared type, using the first few bytes only (cheap on CPU).
- **Check the declared size** before reading the body. Cap at 10 MB per asset to start.
- **Store under server-generated keys only.** Never use the user's file name in a storage key.
- **Serve with:** the stored `Content-Type`, `X-Content-Type-Options: nosniff`, `Content-Disposition: attachment` for anything that isn't an image, and `Content-Security-Policy: default-src 'none'` on all responses that serve user files.
- **Do not render uploaded images in the app's own origin** except through an `<img>` element. Serve user files from a separate subdomain if you can, so a bug can't reach the app's session cookies.

Limitation to state in the plan: a valid PNG can still contain unusual data, and without decoding on the server there is no full check. The mitigations above keep this low-risk. The full fix is server-side scanning or re-encoding, which needs a paid or self-hosted service.

8.5 PSD and complex formats

PSD import is done entirely in the browser. The PSD is parsed by the engine on the user's device, and only the resulting `.zph` is saved to the server. The server never parses a PSD. This is a good outcome for both cost and security: a malicious PSD can only affect the user who opened it.

Keep this rule in place even if a server-side option is added later.

8.6 Request safety

- Parameterized queries only (D1 `prepare` with `bind`).
- JSON bodies validated against schemas, with unknown fields rejected.
- CORS restricted to the production and staging web origins.
- Security headers on the app: strict CSP (no inline scripts), HSTS, `frame-ancestors 'none'`, `Referrer-Policy: strict-origin-when-cross-origin`.
- Errors return stable codes and safe messages. Stack traces are logged, never returned.

8.7 Secrets

- Stored as Workers secrets. Never in the repository, the build output, or the client bundle.
- `ZPH_MASTER_KEY`, `MUSE_API_KEY`, `EMAIL_API_KEY`, `GOOGLE_CLIENT_SECRET`, `TURNSTILE_SECRET`.
- Rotate on a schedule, and immediately if exposed. Scan the repository with a secret scanner in CI.

8.8 Logging

- Log: request ID, route, status, latency, user ID (internal), error code.
- Never log: tokens, cookies, magic links, emails in full, design content, uploads, critique images or text.
- Workers logs have their own retention. Check it and record it in the privacy notice.

---

9. Critique on Muse

Critique is on, funded by the 150-credit balance. The design follows the MVP spec (`zenopher-backend-mvp.md`, Section 6), adapted to D1.

9.1 Checks before the first live request

I have not verified Muse's API. Confirm before enabling:

1. Image input is accepted. If not, critique runs on a text summary of the measurements.
2. Cost per review (`c`), measured from 5 test calls.
3. Whether the API reports remaining credits. If not, the budget row in D1 is the record.
4. Expiry and carry-over of credits.
5. Data retention and training terms for submitted images, in writing.
6. Rate limits on the key.

9.2 Budget

- `starting_credits` = 150, `reserve_credits` = 10, set in config.
- Usable reviews total = `floor((150 - 10) / c)`.
- Per-user limit: 2 reviews per day, to start.

9.3 Reservation, in one statement

Reserve atomically. A single conditional update on the `budget` row either reserves the credits or changes nothing:

```sql
UPDATE budget
SET reserved_credits = reserved_credits + ?1, updated_at = ?2
WHERE id = 1
  AND starting_credits - spent_credits - reserved_credits - reserve_credits >=
```

Check that the statement changed one row. If it changed none, the budget is exhausted and the request is refused. Two requests can't both succeed on the last credits, because SQLite serializes writes.

Settle after the Muse call:

```sql
UPDATE budget
SET reserved_credits = reserved_credits - ?reserved,
    spent_credits    = spent_credits + ?actual,
    updated_at       = ?now
WHERE id = 1;
```

On failure or timeout, release the reservation with the same update and `?actual = 0`.

9.4 Flow

1. Client sends the page PNG (and style name, if set). Sent only after the user has opted in to critique.
2. Worker checks: session, verified email, consent granted (from `consents`), per-user daily limit, PNG signature, dimensions, byte size (cap 4 MB), and the feature flag.
3. Worker computes the SHA-256 of the image. A cache hit in `critique_cache` (24 hours) returns the stored text and costs nothing.
4. Worker reserves credits (9.3). On refusal, returns `feature_disabled` with a message.
5. Worker calls Muse with a 30-second timeout. Images are sent in the request body and never stored, except in the request's memory.
6. Worker validates the response, caps its length, strips markup, stores the text in `critique_cache`, settles the reservation, and returns the text.

Because the call happens in the request, there is no queue and no polling. The client shows a loading state with a cancel button. If the call exceeds the timeout, the reservation is released and the user is told to try again.

If the free plan's request duration makes a 30-second call unreliable, switch to a queued flow with cron processing, and keep the same reservation logic.

9.5 Security specific to critique

- The Muse key is a Worker secret. The client never sees it.
- Output is untrusted text: displayed as plain text, length-capped, never parsed into commands, and never able to change a design.
- Text inside a design can try to instruct the model. The output format is fixed, so the worst outcome is a misleading comment.

---

10. Core flows

10.1 Sign-in with magic link

1. Client posts the email, with a Turnstile token.
2. Worker verifies the Turnstile token, applies rate limits, and creates an `auth_tokens` row (hashed, 15 minutes).
3. Worker sends the link by email. The response is the same whether or not the account exists.
4. The link opens the app with the token. Worker checks it, marks it used, creates the user if needed, creates a session, and sets the refresh cookie.

10.2 Save a version

1. Client sends `base_version`, the new content hash, and the document as gzip bytes.
2. Worker checks the session, ownership or edit grant, rate limit (12 per minute per design), and body size (cap 25 MB, to start).
3. Worker fetches the design's DEK (unwrapped with the master key) and encrypts the blob.
4. Worker writes the blob to R2 under a server-generated key.
5. Worker runs a D1 batch:
   - `UPDATE designs SET current_version = ?new, updated_at = ? WHERE id = ? AND current_version = ?base`
   - if one row changed, `INSERT INTO design_versions ...`
   - if none changed, the batch is abandoned and the response is `409`
6. On `409`, the response includes the current version number. The client follows the conflict flow.
7. An orphaned R2 object (written but not referenced) is removed by the cron job after 24 hours.

Step 5 must run as a single batch. Otherwise a version row could be inserted without the design pointer moving.

10.3 Open a design

1. Worker checks access.
2. Worker reads the version's `r2_key`, fetches the blob, decrypts it, and streams it to the client with `Cache-Control: private, no-store`.
3. The client caches the document locally, keyed by version number. If the cached version matches `current_version` on the next open, the client skips the download.

10.4 Offline editing and replay

Unchanged from the full design (`zenopher-backend.md`, 8.7). The client keeps the latest pending save per design in IndexedDB, replays it with `base_version`, and handles `409` as a conflict copy. No server cost while offline.

10.5 Trash, restore, purge

- **Trash:** sets `trashed_at`, revokes share grants for the design.
- **Restore:** clears `trashed_at`. Shares stay revoked.
- **Purge (cron, daily, batched):** selects designs trashed more than 30 days ago, in batches of 50. For each: delete `wrapped_dek`, delete R2 blobs in batches, delete version rows, delete the design row, and log the completion.

Deleting the wrapped key first means a purge interrupted halfway still leaves the remaining blobs unreadable.

10.6 Thumbnails

The client renders the thumbnail and uploads it as a PNG asset linked to the design. The server checks the PNG signature, dimensions (for example maximum 512 px), and size (maximum 200 KB), then encrypts and stores it under the design's DEK. This avoids all server-side image processing.

10.7 Assets (images and fonts)

1. Client uploads the file to the Worker, which checks the declared type and size before reading the body.
2. Worker checks the magic bytes, encrypts the file under the owner's design DEK or a per-asset DEK, and stores it in R2.
3. Asset status becomes `ready`. Designs can only reference `ready` assets.
4. Serving follows the rules in Section 8.4.

10.8 Sharing

- **Link share:** owner creates a grant with a random token (stored hashed). The token is shown once. Recipients open the link; the Worker checks the hash, expiry, revocation, and that the design isn't trashed.
- **User share:** grant by account ID, with a permission level. Requires the grantee to have a verified account.

- **Revocation** takes effect on the next request.
- Shared views never include the owner's email.

10.9 Account export

1. User requests an export. Worker inserts a `jobs` row (kind `export`).
2. The cron job processes export jobs in small batches, because decrypting many designs can exceed the per-run CPU limit. Each batch processes a few designs and records progress.
3. Output is a ZIP built from the decrypted versions, profile data, settings, consent history, and learning profile. Critique history is included as metadata only.
4. The ZIP is encrypted with a per-export key, stored in R2 with a 7-day lifetime, and the owner is emailed a sign-in-protected link. The email contains no data.

If a single large design exceeds the per-run limit, the export includes a note and provides that design as separate downloads.

10.10 Account deletion

1. User requests deletion and confirms with a fresh sign-in.
2. Worker sets `status = pending_deletion`, `deletion_at = now + 14 days`, revokes all sessions and grants, and stops processing.
3. Cancellation: sign in during the grace period to reset the status.
4. After the grace period, cron processes the deletion in batches:
    - delete wrapped keys for all of the user's designs (crypto-shredding)
    - delete R2 blobs and assets
    - delete learning events via `learning_link`, then `learning_link`
    - anonymize critique records (remove the user hash, keep counts)
    - delete `oauth_links`, `sessions`, `auth_tokens`, and the `users` row
    - keep `consents` only as the legal record, with the user ID replaced by a random value

10.11 Consent

- Granting or withdrawing consent appends a row to `consents`.
- Withdrawal takes effect immediately. The Worker checks the latest row for each purpose on every relevant request, not a cached copy.
- Withdrawing learning consent queues deletion of that user's events.

10.12 Learning events

1. Client records events only if consent is granted, batches them every 60 seconds or 50 events, and sends them.
2. Worker checks consent from `consents`, validates each event against a schema, maps the user to `pseudo_id` via `learning_link`, and inserts rows with no user ID.

3. Cron aggregates events daily into a small summary table and deletes raw events older than 12 months.

10.13 Catalog

- Styles, presets, and tutorials are versioned JSON files in the repository. CI validates them, including that every measurement name exists in the engine.
- Publishing is a pull request with a second reviewer.
- The Worker serves the catalog from its bundled files or from R2, with `ETag` and long cache lifetimes. Clients cache by hash.
- Designs stay pinned to the style version they were checked against.

---

11. Rate limits

Route	Limit	Scope
Magic-link request	5 per 15 minutes	Per email and per hashed IP
Token verification	10 per 15 minutes	Per hashed IP
Refresh	30 per minute	Per session
Save version	12 per minute	Per design
Design list	60 per minute	Per user
Asset upload	20 per hour	Per user
Share creation	20 per hour	Per user
Critique	2 per day	Per user, plus the budget
Learning events	6 per minute	Per user

Limits are enforced with the platform's rate limiting if available, otherwise with `rate_counters` in D1. Auth and critique fail closed. Reads fail open.

Responses include `429` with `Retry-After` when a limit is hit.

---

12. Traffic and efficiency

- **Autosave:** 3 seconds after the last change, on blur, and on close. Nothing is sent if the content hash hasn't changed.
- **Caching:** the catalog and design metadata are cached on the client with `ETag` revalidation. Catalog checks happen at most once a day.
- **Lists** return metadata only, with cursor pagination.

- **Thumbnails** are fetched once per version.
- **D1 writes** are batched. Learning events are batched. Cron work is batched.
- **No polling** in the MVP, since critique is synchronous.
- **Compression:** gzip for responses. Document blobs are compressed before encryption; images aren't re-compressed.

Watch the daily request and D1 operation counts. If either approaches its cap, the first lever is client-side batching, not more infrastructure.

---

13. Observability and operations

- **Health:** a `/health` route that checks D1 and R2 reachability, with no internal detail in the response.
- **Monitoring:** the platform's analytics for request counts and errors, plus a free uptime checker on `/health`.
- **Alerts** (email, from the monitoring tool): error rate above 2% over 15 minutes, Muse budget at 80% and 100%, D1 or R2 approaching their free limits, cron failures.
- **Error tracking:** a free tier of an error tracker, with request bodies and headers scrubbed.
- **Runbook:** how to disable critique (config flag), how to revoke all sessions (one SQL update), how to rotate the master key (Section 7.4), and how to restore from the weekly export.

---

14. Backups and recovery

- **D1:** use the platform's built-in point-in-time restore if it's available on the free plan, and check the retention window. Record the window in the privacy notice.
- **Weekly export:** a cron job exports the metadata tables as JSON, encrypts the file with a key stored separately from the export, and writes it to R2. Keep four copies.
- **Design blobs** are in R2 with versioning enabled, so accidental overwrites are reversible.
- **Restore drill:** once a month, restore the latest export into a scratch D1 database, decrypt a sample design, and record the time taken.
- **Recovery targets:** up to 7 days of metadata in the worst case (the weekly export), with browser copies covering recent work.

---

15. Failure behavior

Failure	Behavior
D1 unavailable	Saves fail with `503`. The client keeps the local copy and retries. Cached designs still open.
R2 unavailable	Saves and opens fail with `503`. Local copies still work.
Muse down or timing out	Reservation is released. Critique shows "unavailable". Rules checks unaffected.
Email provider down	Magic links fail with a clear message. Google sign-in still works.
Budget exhausted	Critique paused for everyone. Clear message.
Daily platform cap reached	Service degrades for everyone until reset. Offline editing continues.
Cron job fails	Batches are idempotent, so the next run resumes safely.
Partial write (blob without row)	Cron removes orphaned blobs after 24 hours.

---

16. Limits of this setup

- **Not suitable for high traffic.** Free daily caps will limit growth. Plan the paid upgrade when the caps are reached, not before.
- **Server-side protection is thinner.** Without re-encoding or scanning, uploads rely on type checks and safe serving. Acceptable for a closed test, not for an open public launch.
- **No real-time collaboration.** It needs a different architecture (Durable Objects or a paid tier).
- **The master key is held by the Worker.** A compromised Worker exposes data. This is accepted for the MVP and stated in the privacy notice.
- **Data location** is determined by the platform, not chosen by you. Confirm with counsel whether this is acceptable under the DPDP Act and your users' expectations.
- **Penetration test** is not included. Do it before opening sign-up to the public.

---

17. Phases

1. **Foundation:** Worker skeleton, D1 migrations, magic-link and Google sign-in, sessions, design create and save with version checks, encryption, authorization tests. *Gate:* a

save and reload round trip works through encryption, a deliberate conflict returns `409`, and authorization tests pass.

2. **Designs complete:** open, trash, restore, cron purge, thumbnails, assets, offline replay in the client. *Gate:* a purged design's blobs can't be decrypted, and an offline edit on two devices produces a conflict copy.
3. **Sharing and catalog:** link shares, user grants, revocation, catalog files with CI validation. *Gate:* a revoked link stops working on the next request.
4. **Critique on Muse:** consent check, budget reservation, cache, Muse adapter, timeouts. *Gate:* concurrent test requests can't overspend the budget, and the cutoff stops calls at the limit.
5. **Learning and privacy:** consent ledger, event ingestion, aggregation, export, account deletion. *Gate:* withdrawing consent stops ingestion at once, and a deleted account can't be recovered from the weekly export.
6. **Launch readiness:** security review, authorization test run, traffic test within free caps, privacy notice and terms reviewed by counsel. *Gate:* all high findings closed, notices approved, and monitoring alerts tested.

---

18. Decisions

#	Decision	Choice	Notes
1	API	TypeScript on Cloudflare Workers	Rust is possible but the free plan's limits favor TypeScript
2	Database	Cloudflare D1	Check free limits and time-travel retention
3	Storage	Cloudflare R2	
4	Sign-in	Magic link and Google. No passwords in MVP	Avoids CPU-heavy hashing
5	Encryption	Envelope with Web Crypto AES-GCM, master key as Worker secret	Not end-to-end
6	Heavy processing	In the browser (PSD, thumbnails, re-encoding)	Server does none
7	Malware scanning	None on server. Type checks, size caps, safe serving	Stated limitation
8	Critique	Synchronous, on Muse, 150 credits, 10-credit reserve	Verify Muse first
9	Jobs	Cron plus a D1 table	No separate queue
10	Frontend	Cloudflare Pages preferred	Vercel Hobby is non-commercial
11	Email	Free tier of a transactional provider	Daily cap
12	Penetration test	Before public sign-up	Paid, deferred
13	Version retention	30 days	Storage cap

19. Changes from the MVP spec

Area	MVP spec	This document
API language	Rust binary	TypeScript Worker
Database	Managed Postgres	D1
Queue	Postgres `SKIP LOCKED`	Cron plus D1 job table
Rate limits	Postgres counters	Platform limiter, or D1 counters
Scanner	ClamAV	None. Type checks and re-encoding in the browser
PSD import	Server parse behind a flag	Browser parse, server stores only `.zph`
Thumbnails	Worker renders	Browser renders and uploads
Passwords	Argon2id	None in MVP
Critique	Worker job with polling	Synchronous call with timeout
Backups	Daily `pg_dump`	Weekly encrypted D1 export, plus platform restore if available
Key	Secret store, behind `KeyProvider`	Worker secret, Web Crypto

20. Open items to verify

1. Current free-tier limits: Workers requests and CPU time, D1 reads, writes, and storage, R2 operations and storage, and cron frequency.
2. Whether rate limiting is available on the free plan, or D1 counters are needed.
3. D1 time-travel retention on the free plan.
4. Muse: image input, cost per review, balance reporting, retention, and training terms.
5. Email provider free-tier daily limit.
6. Counsel review of data location, retention, notices, and the grievance process.